

```
!pip install -q category_encoders catboost optuna imbalanced-learn
```

```

85.9/85.9 kB 1.5 MB/s eta 0:00:00
97.1/97.1 MB 7.2 MB/s eta 0:00:00
425.6/425.6 kB 17.5 MB/s eta 0:00:00

```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
%cd "/content/drive/MyDrive/competition"
```

/content/drive/MyDrive/competition

✧ Import Libraries

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
import warnings
import json
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_auc_score
from sklearn.preprocessing import LabelEncoder
from scipy.stats import rankdata
import xgboost as xgb
import lightgbm as lgb
import catboost as cb
import optuna
import shutil
from google.colab import files
from pathlib import Path

```

✧ Setup & Data Loading

```

SEED = 42
np.random.seed(SEED)
optuna.logging.set_verbosity(optuna.logging.WARNING)
warnings.filterwarnings("ignore")

```

```

PATH = Path.cwd() / "input"
train_file = PATH / "train.csv"
test_file = PATH / "test.csv"
sample_sub_file = PATH / "sample_submission.csv"

if train_file.exists() and test_file.exists() and sample_sub_file.exists():

```

```

print("All files exist and path is correctly set")
else:
    print("Some files are missing or path is not correctly set")

```

All files exist and path is correctly set

```

PATH = Path.cwd() / "input"

train = pd.read_csv(PATH / "train.csv")
test = pd.read_csv(PATH / "test.csv")
sample_sub = pd.read_csv(PATH / "sample_submission.csv")

os.makedirs("output", exist_ok=True)

print(f"Train shape: {train.shape}")
print(f"Test shape: {test.shape}")
print(f"Sample sub : {sample_sub.shape}")
print(f"\nTarget distribution:\n{train['Drafted'].value_counts()}")
print(f"\nDraft rate: {train['Drafted'].mean():.4f}")

```

```

Train shape: (2781, 16)
Test shape: (696, 15)
Sample sub : (696, 2)

Target distribution:
Drafted
1.0    1803
0.0     978
Name: count, dtype: int64

Draft rate: 0.6483

```

✓ Preprocessing & Feature Engineering

```

def prepare_features(train_df, test_df):
    tr = train_df.copy()
    te = test_df.copy()

    y = tr["Drafted"].values

    athletic_cols = [
        "Age", "Sprint_40yd", "Vertical_Jump", "Bench_Press_Reps",
        "Broad_Jump", "Agility_3cone", "Shuttle"
    ]
    for col in athletic_cols:
        tr[f"{col}_missing"] = tr[col].isnull().astype(int)
        te[f"{col}_missing"] = te[col].isnull().astype(int)

    numeric_cols = tr.select_dtypes(include=[np.number]).columns.tolist()
    numeric_cols = [c for c in numeric_cols if c not in ("Id", "Drafted")]

    train_means = tr[numeric_cols].mean()
    tr[numeric_cols] = tr[numeric_cols].fillna(train_means)
    te[numeric_cols] = te[numeric_cols].fillna(train_means)

    school_counts = tr["School"].value_counts().to_dict()

```

```

tr["School_count"] = tr["School"].map(school_counts).fillna(0)
te["School_count"] = te["School"].map(school_counts).fillna(0)

cat_cols = ["Player_Type", "Position_Type", "Position"]
label_encoders = {}
for col in cat_cols:
    le = LabelEncoder()
    le.fit(tr[col])
    tr[col] = le.transform(tr[col])
    te_vals = te[col].copy()
    te[col] = te_vals.map(
        lambda x, _le=le: _le.transform([x])[0] if x in _le.classes_ else -1
    )
    label_encoders[col] = le

for df in [tr, te]:
    df["BMI"] = df["Weight"] / (df["Height"] ** 2)

    df["SPEED_SCORE"] = (df["Weight"] * 200.0) / (df["Sprint_40yd"] ** 4)

vj_mean, vj_std = tr["Vertical_Jump"].mean(), tr["Vertical_Jump"].std()
bj_mean, bj_std = tr["Broad_Jump"].mean(), tr["Broad_Jump"].std()

for df in [tr, te]:
    vj_z = (df["Vertical_Jump"] - vj_mean) / vj_std
    bj_z = (df["Broad_Jump"] - bj_mean) / bj_std
    df["EXPLOSIVENESS"] = 0.5 * (vj_z + bj_z)

    df["AGILITY_RATIO"] = np.where(
        df["Shuttle"] != 0,
        df["Agility_3cone"] / df["Shuttle"],
        np.nan
    )

agility_median = tr["AGILITY_RATIO"].median()
tr["AGILITY_RATIO"] = tr["AGILITY_RATIO"].fillna(agility_median)
te["AGILITY_RATIO"] = te["AGILITY_RATIO"].fillna(agility_median)

for df in [tr, te]:
    df.replace([np.inf, -np.inf], np.nan, inplace=True)

speed_median = tr["SPEED_SCORE"].median()
tr["SPEED_SCORE"] = tr["SPEED_SCORE"].fillna(speed_median)
te["SPEED_SCORE"] = te["SPEED_SCORE"].fillna(speed_median)

drop_cols = ["Id", "School", "Drafted"]
X_train = tr.drop(columns=[c for c in drop_cols if c in tr.columns])
X_test = te.drop(columns=[c for c in drop_cols if c in te.columns])

remaining_median = X_train.median()
X_train = X_train.fillna(remaining_median)
X_test = X_test.fillna(remaining_median)

feature_names = X_train.columns.tolist()

return X_train, y, X_test, feature_names

```

```
X_train, y_train, X_test, feature_names = prepare_features(train, test)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"\nFeatures ({len(feature_names)}):")
for i, f in enumerate(feature_names):
    print(f" {i+1:2d}. {f}")
```

```
X_train shape: (2781, 25)
```

```
X_test shape: (696, 25)
```

```
Features (25):
```

1. Year
2. Age
3. Height
4. Weight
5. Sprint_40yd
6. Vertical_Jump
7. Bench_Press_Reps
8. Broad_Jump
9. Agility_3cone
10. Shuttle
11. Player_Type
12. Position_Type
13. Position
14. Age_missing
15. Sprint_40yd_missing
16. Vertical_Jump_missing
17. Bench_Press_Reps_missing
18. Broad_Jump_missing
19. Agility_3cone_missing
20. Shuttle_missing
21. School_count
22. BMI
23. SPEED_SCORE
24. EXPLOSIVENESS
25. AGILITY_RATIO

✓ Cross-Validation Strategy

```
def cv_fit_predict(make_model, X, y, X_test, n_splits=5, seed=SEED):
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)

    oof_preds = np.zeros(len(X))
    test_preds = np.zeros(len(X_test))

    X_np = X.values if hasattr(X, "values") else X
    X_test_np = X_test.values if hasattr(X_test, "values") else X_test

    for fold, (train_idx, val_idx) in enumerate(skf.split(X_np, y)):
        X_tr, X_val = X_np[train_idx], X_np[val_idx]
        y_tr, y_val = y[train_idx], y[val_idx]

        model = make_model()

        if isinstance(model, xgb.XGBClassifier):
            model.fit(
                X_tr, y_tr,
```

```

        eval_set=[(X_val, y_val)],
        verbose=False
    )
elif isinstance(model, lgb.LGBMClassifier):
    model.fit(
        X_tr, y_tr,
        eval_set=[(X_val, y_val)],
        callbacks=[lgb.log_evaluation(0)]
    )
elif isinstance(model, cb.CatBoostClassifier):
    model.fit(
        X_tr, y_tr,
        eval_set=(X_val, y_val),
        verbose=0
    )
else:
    model.fit(X_tr, y_tr)

oof_preds[val_idx] = model.predict_proba(X_val)[ :, 1]
test_preds += model.predict_proba(X_test_np)[ :, 1] / n_splits

oof_auc = roc_auc_score(y, oof_preds)
return oof_preds, test_preds, oof_auc

```

✓ Model Diversity & Optuna Tuning

✓ Default Baselines

```

def make_xgb_default():
    return xgb.XGBClassifier(
        n_estimators=500,
        max_depth=6,
        learning_rate=0.05,
        subsample=0.8,
        colsample_bytree=0.8,
        min_child_weight=3,
        reg_alpha=0.1,
        reg_lambda=1.0,
        scale_pos_weight=1.0,
        use_label_encoder=False,
        eval_metric="auc",
        early_stopping_rounds=50,
        random_state=SEED,
        n_jobs=-1
    )

oof_xgb_def, test_xgb_def, auc_xgb_def = cv_fit_predict(
    make_xgb_default, X_train, y_train, X_test
)
print(f"XGBoost (default) OOF AUC: {auc_xgb_def:.6f}")

```

XGBoost (default) OOF AUC: 0.825679

```
def make_lgb_default():
    return lgb.LGBMClassifier(
        n_estimators=500,
        max_depth=6,
        learning_rate=0.05,
        subsample=0.8,
        colsample_bytree=0.8,
        min_child_samples=10,
        reg_alpha=0.1,
        reg_lambda=1.0,
        metric="auc",
        random_state=SEED,
        n_jobs=-1,
        verbosity=-1
    )

oof_lgb_def, test_lgb_def, auc_lgb_def = cv_fit_predict(
    make_lgb_default, X_train, y_train, X_test
)
print(f"LightGBM (default) OOF AUC: {auc_lgb_def:.6f}")
```

LightGBM (default) OOF AUC: 0.815310

```
def make_cb_default():
    return cb.CatBoostClassifier(
        iterations=500,
        depth=6,
        learning_rate=0.05,
        l2_leaf_reg=3.0,
        random_seed=SEED,
        eval_metric="AUC",
        early_stopping_rounds=50,
        verbose=0
    )

oof_cb_def, test_cb_def, auc_cb_def = cv_fit_predict(
    make_cb_default, X_train, y_train, X_test
)
print(f"CatBoost (default) OOF AUC: {auc_cb_def:.6f}")
```

CatBoost (default) OOF AUC: 0.829207

✓ Optuna Hyperparameter Tuning

```
def catboost_objective(trial):
    params = {
        "iterations": trial.suggest_int("iterations", 300, 1500),
        "depth": trial.suggest_int("depth", 4, 10),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3, log=True),
        "l2_leaf_reg": trial.suggest_float("l2_leaf_reg", 1e-3, 10.0, log=True),
        "bagging_temperature": trial.suggest_float("bagging_temperature", 0.0, 5.0),
        "random_strength": trial.suggest_float("random_strength", 0.0, 5.0),
        "border_count": trial.suggest_int("border_count", 32, 255),
        "min_data_in_leaf": trial.suggest_int("min_data_in_leaf", 1, 50),
        "random_seed": SEED,
        "eval_metric": "AUC",
    }
```

```

        "early_stopping_rounds": 50,
        "verbose": 0,
    }

    def make_model():
        return cb.CatBoostClassifier(**params)

    _, _, oof_auc = cv_fit_predict(make_model, X_train, y_train, X_test)
    return oof_auc

study_cb = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.TPESampler(seed=SEED)
)
study_cb.optimize(catboost_objective, n_trials=40, show_progress_bar=True)

print(f"\nBest CatBoost trial AUC: {study_cb.best_value:.6f}")
print(f"Best params: {json.dumps(study_cb.best_params, indent=2)}")

```

Best trial: 21. Best value: 0.834316: 100%

40/40 [02:52<00:00, 3.28s/it]

Best CatBoost trial AUC: 0.834316

```

Best params: {
    "iterations": 957,
    "depth": 5,
    "learning_rate": 0.16442164654522043,
    "l2_leaf_reg": 0.5634793228387781,
    "bagging_temperature": 4.3334921952467065,
    "random_strength": 4.371155803697163,
    "border_count": 156,
    "min_data_in_leaf": 45
}

```

```

def xgboost_objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 300, 1500),
        "max_depth": trial.suggest_int("max_depth", 3, 10),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3, log=True),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.3, 1.0),
        "min_child_weight": trial.suggest_int("min_child_weight", 1, 20),
        "reg_alpha": trial.suggest_float("reg_alpha", 1e-4, 10.0, log=True),
        "reg_lambda": trial.suggest_float("reg_lambda", 1e-4, 10.0, log=True),
        "gamma": trial.suggest_float("gamma", 0.0, 5.0),
        "use_label_encoder": False,
        "eval_metric": "auc",
        "early_stopping_rounds": 50,
        "random_state": SEED,
        "n_jobs": -1,
    }

    def make_model():
        return xgb.XGBClassifier(**params)

    _, _, oof_auc = cv_fit_predict(make_model, X_train, y_train, X_test)
    return oof_auc

```

```

study_xgb = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.TPESampler(seed=SEED)
)
study_xgb.optimize(xgboost_objective, n_trials=40, show_progress_bar=True)

print(f"\nBest XGBoost trial AUC: {study_xgb.best_value:.6f}")
print(f"Best params: {json.dumps(study_xgb.best_params, indent=2)}")

```

Best trial: 31. Best value: 0.837212: 100%

40/40 [00:32<00:00, 1.45it/s]

```

Best XGBoost trial AUC: 0.837212
Best params: {
  "n_estimators": 769,
  "max_depth": 3,
  "learning_rate": 0.10311891709982446,
  "subsample": 0.718220554491657,
  "colsample_bytree": 0.39001125599829034,
  "min_child_weight": 4,
  "reg_alpha": 0.00022911767455336462,
  "reg_lambda": 0.024230260333881596,
  "gamma": 1.7833925818760608
}

```

```

def lightgbm_objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 300, 1500),
        "max_depth": trial.suggest_int("max_depth", 3, 12),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3, log=True),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.3, 1.0),
        "min_child_samples": trial.suggest_int("min_child_samples", 5, 50),
        "reg_alpha": trial.suggest_float("reg_alpha", 1e-4, 10.0, log=True),
        "reg_lambda": trial.suggest_float("reg_lambda", 1e-4, 10.0, log=True),
        "num_leaves": trial.suggest_int("num_leaves", 15, 127),
        "min_split_gain": trial.suggest_float("min_split_gain", 0.0, 5.0),
        "metric": "auc",
        "random_state": SEED,
        "n_jobs": -1,
        "verbosity": -1,
    }

    def make_model():
        return lgb.LGBMClassifier(**params)

    _, _, oof_auc = cv_fit_predict(make_model, X_train, y_train, X_test)
    return oof_auc

study_lgb = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.TPESampler(seed=SEED)
)
study_lgb.optimize(lightgbm_objective, n_trials=40, show_progress_bar=True)

print(f"\nBest LightGBM trial AUC: {study_lgb.best_value:.6f}")
print(f"Best params: {json.dumps(study_lgb.best_params, indent=2)}")

```

Best trial: 34. Best value: 0.833084: 100%

40/40 [01:54<00:00, 4.66s/it]

Best LightGBM trial AUC: 0.833084

```
Best params: {
  "n_estimators": 1166,
  "max_depth": 12,
  "learning_rate": 0.0208419891479453,
  "subsample": 0.7017502737723398,
  "colsample_bytree": 0.862449876837623,
  "min_child_samples": 45,
  "reg_alpha": 0.0004864905630186642,
  "reg_lambda": 0.0004387045362864256,
  "num_leaves": 55,
  "min_split_gain": 2.0763134774836653
}
```

✓ Validation Check — Tuned vs Default

```
best_cb_params = study_cb.best_params.copy()
best_cb_params.update({"random_seed": SEED, "eval_metric": "AUC",
                      "early_stopping_rounds": 50, "verbose": 0})

def make_cb_tuned():
    return cb.CatBoostClassifier(**best_cb_params)

oof_cb_tuned, test_cb_tuned, auc_cb_tuned = cv_fit_predict(
    make_cb_tuned, X_train, y_train, X_test
)

best_xgb_params = study_xgb.best_params.copy()
best_xgb_params.update({"use_label_encoder": False, "eval_metric": "auc",
                      "early_stopping_rounds": 50, "random_state": SEED, "n_jobs": -1})

def make_xgb_tuned():
    return xgb.XGBClassifier(**best_xgb_params)

oof_xgb_tuned, test_xgb_tuned, auc_xgb_tuned = cv_fit_predict(
    make_xgb_tuned, X_train, y_train, X_test
)

best_lgb_params = study_lgb.best_params.copy()
best_lgb_params.update({"metric": "auc", "random_state": SEED,
                      "n_jobs": -1, "verbosity": -1})

def make_lgb_tuned():
    return lgb.LGBMClassifier(**best_lgb_params)

oof_lgb_tuned, test_lgb_tuned, auc_lgb_tuned = cv_fit_predict(
    make_lgb_tuned, X_train, y_train, X_test
)

print("Tuned vs Default Comparison:")

if auc_cb_tuned > auc_cb_def:
    test_cb_best, auc_cb_best = test_cb_tuned, auc_cb_tuned
    print(f" CatBoost : Tuned WINS  ({auc_cb_tuned:.6f} > {auc_cb_def:.6f})")
```

```

else:
    test_cb_best, auc_cb_best = test_cb_def, auc_cb_def
    print(f" CatBoost : Default WINS ({auc_cb_def:.6f} >= {auc_cb_tuned:.6f})")

if auc_xgb_tuned > auc_xgb_def:
    test_xgb_best, auc_xgb_best = test_xgb_tuned, auc_xgb_tuned
    print(f" XGBoost : Tuned WINS ({auc_xgb_tuned:.6f} > {auc_xgb_def:.6f})")
else:
    test_xgb_best, auc_xgb_best = test_xgb_def, auc_xgb_def
    print(f" XGBoost : Default WINS ({auc_xgb_def:.6f} >= {auc_xgb_tuned:.6f})")

if auc_lgb_tuned > auc_lgb_def:
    test_lgb_best, auc_lgb_best = test_lgb_tuned, auc_lgb_tuned
    print(f" LightGBM : Tuned WINS ({auc_lgb_tuned:.6f} > {auc_lgb_def:.6f})")
else:
    test_lgb_best, auc_lgb_best = test_lgb_def, auc_lgb_def
    print(f" LightGBM : Default WINS ({auc_lgb_def:.6f} >= {auc_lgb_tuned:.6f})")

print(f"\n Final CatBoost AUC: {auc_cb_best:.6f}")
print(f" Final XGBoost AUC: {auc_xgb_best:.6f}")
print(f" Final LightGBM AUC: {auc_lgb_best:.6f}")

```

Tuned vs Default Comparison:

```

CatBoost : Tuned WINS (0.834316 > 0.829207)
XGBoost : Tuned WINS (0.837212 > 0.825679)
LightGBM : Tuned WINS (0.833084 > 0.815310)

```

```

Final CatBoost AUC: 0.834316
Final XGBoost AUC: 0.837212
Final LightGBM AUC: 0.833084

```

✓ Ensembling via Rank Averaging

```

def rank_average_ensemble(*pred_arrays):
    n = len(pred_arrays[0])
    rank_sum = np.zeros(n)

    for preds in pred_arrays:
        ranks = rankdata(preds, method="average")
        rank_sum += ranks

    rank_avg = rank_sum / len(pred_arrays)
    rank_avg = (rank_avg - rank_avg.min()) / (rank_avg.max() - rank_avg.min())
    return rank_avg

final_preds = rank_average_ensemble(test_cb_best, test_xgb_best, test_lgb_best)

print(f"Prediction range: [{final_preds.min():.6f}, {final_preds.max():.6f}]")
print(f"Prediction mean : {final_preds.mean():.6f}")

```

```

Prediction range: [0.000000, 1.000000]
Prediction mean : 0.497106

```

Submission

```
submission = sample_sub.copy()
submission["Drafted"] = final_preds

submission.to_csv("output/submission.csv", index=False)

print(f"Submission shape: {submission.shape}")
print(f"\nHead:\n{submission.head(10)}")
print(f"\nTail:\n{submission.tail(5)}")
print(f"\nDrafted distribution stats:")
print(submission["Drafted"].describe())
```

Submission shape: (696, 2)

Head:

	Id	Drafted
0	2781	0.374819
1	2782	0.771828
2	2783	0.932947
3	2784	0.864930
4	2785	0.754462
5	2786	0.232996
6	2787	0.058852
7	2788	0.315002
8	2789	0.523878
9	2790	0.914134

Tail:

	Id	Drafted
691	3472	0.105644
692	3473	0.460203
693	3474	0.634346
694	3475	0.178003
695	3476	0.395562

Drafted distribution stats:

count	696.000000
mean	0.497106
std	0.287692
min	0.000000
25%	0.251327
50%	0.498794
75%	0.745779
max	1.000000

Name: Drafted, dtype: float64

